
Question 3

What is the difference between these two?

```
java
    .hasRole("MANAGER")
```

and

```
java
    .hasAnyRole("STAFF", "MANAGER")
```

Question 4

Why don't we send the username and password with every protected API request after login?

Question 5 — ★ Most important

Imagine an interviewer asks:

""Explain how you implemented JWT authentication and role-based authorization in your Spring Boot project.""

Give me a **2–3 minute** interview answer in your own words.

Don't worry about grammar. **Explain it naturally like you are sitting in an interview.**

I'll evaluate your answer and turn it into your final **Day 5 interview-ready answer.**

YOU

ok now give me complete notes of today whatever i learned of springboot for interview purpose and give all answers of interview ready questions

Day 5 — Spring Boot Complete Interview Notes

JWT Authentication + SecurityContext + Role-Based Authorization

Preparation Plan: Day 5

Session: 7:00 AM – 9:00 AM

Project: Spring Boot Staff Training Project

Today you continued directly from Day 4. The main goal was to finish the unfinished JWT filter, connect it to Spring Security, protect Staff APIs, and implement/test **STAFF vs MANAGER authorization**. This matches the Day 5 sequence from your previous notes.

1. 🎯 What You Learned Today

Today you covered:

1. `JwtAuthenticationFilter``
 2. `OncePerRequestFilter``
 3. Authorization header
 4. Bearer token
 5. JWT extraction
 6. `substring(7)``
 7. Username extraction from JWT
 8. `UserDetailsService`` inside JWT authentication
 9. JWT validation
 10. `Authentication`` object
 11. `SecurityContext``
 12. `SecurityContextHolder``
 13. `SecurityFilterChain``
 14. `addFilterBefore()``
 15. Protected APIs
 16. Authentication vs Authorization
 17. Roles and authorities
 18. `.roles()``
 19. `ROLE_STAFF`` / `ROLE_MANAGER``
 20. `.hasRole()``
 21. `.hasAnyRole()``
 22. Role-based authorization
 23. Testing authorized/unauthorized requests
 24. STAFF vs MANAGER permissions
 25. Debugging duplicate usernames
-

2. 🔥 Complete Security Architecture

Your project now has:

text

```
CLIENT / POSTMAN
↓
Spring Security
↓
JwtAuthenticationFilter
↓
Extract JWT
↓
Validate JWT
↓
Extract Username
↓
CustomUserDetailsService
↓
UserRepository
↓
MySQL
↓
UserDetails
↓
Authentication
↓
SecurityContext
↓
Authorization
↓
STAFF / MANAGER
↓
Controller
↓
Service
↓
Repository
↓
MySQL
```

This extends the architecture from Day 4, where security was added around your existing Controller → Service → Repository → JPA/Hibernate → MySQL application.

3. What is `JwtAuthenticationFilter`?

Your class:

```
java

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter
```

The filter is responsible for processing the JWT attached to incoming requests.

Main responsibilities

text

```

Authorization Header
↓
Extract Bearer Token
↓
Extract Username
↓
Load UserDetails
↓
Validate JWT
↓
Create Authentication
↓
SecurityContext

```

These are the six responsibilities identified in your Day 4 notes. NfileciteÖturn0file0ÖL411-L432Ô

Interview answer

""JwtAuthenticationFilter is a custom Spring Security filter that extracts the Bearer JWT from the Authorization header, validates it, loads the user's details, creates an Authentication object with the user's authorities, and stores that authentication in the SecurityContext.""

4. What is `OncePerRequestFilter`?

You extended:

```

java
OncePerRequestFilter

```

It provides a convenient base class for implementing a filter that should execute once for a request.

Your JWT logic is implemented inside:

```

java
doFilterInternal()

```

Interview answer

""`OncePerRequestFilter` is a Spring-provided base class used to implement filters that are executed once per request. We extend it to implement our JWT authentication logic.""

5. Authorization Header

The client sends the JWT through:

```
http

Authorization: Bearer <JWT>
```

Your filter reads it using:

```
java

String authHeader = request.getHeader("Authorization");
```

For example:

```
text

Authorization:
Bearer eyJhbGciOiJIUzI1NiJ9...
```

Your Day 4 notes specifically identified the Bearer token format as the way the client sends the JWT in the Authorization header.

6. What is Bearer?

`Bearer` is the authentication scheme used in the Authorization header.

```
http

Authorization: Bearer <JWT>
```

Interview answer

""Bearer is the authentication scheme used in the Authorization header to indicate that the following value is an authentication token.""

Remember

Bearer = "Here is the token that I am presenting for authentication."

7. Checking the Authorization Header

Your code:

```
java

if (authHeader == null || !authHeader.startsWith("Bearer ")) {
    filterChain.doFilter(request, response);
    return;
}
```

This checks:

Condition 1

```
java

authHeader == null
```

No Authorization header exists.

Condition 2

```
java

!authHeader.startsWith("Bearer ")
```

The Authorization header exists, but it isn't using the Bearer scheme.

Example:

```
text

Authorization: Basic xyz
```

instead of:

```
text

Authorization: Bearer JWT
```

Interview answer

""We check whether the Authorization header starts with `Bearer ` to make sure the request contains a Bearer JWT before attempting to extract and validate the token.""

8. Extracting the JWT

Your code:

```
java
String token = authHeader.substring(7);
```

Suppose:

```
text
Bearer abc123xyz
```

The string:

```
text
Bearer
```

contains **7 characters**, including the space.

Therefore:

```
java
substring(7)
```

removes the prefix.

Result:

```
text
abc123xyz
```

Important distinction

`substring(7)` does NOT extract the username.

It extracts the JWT.

Then:

```
java
String username = jwtService.extractUsername(token);
```

extracts the username from the JWT.

Interview answer

""`substring(7)` removes the `Bearer ` prefix from the Authorization header and gives us only the JWT. The username is then extracted separately from the JWT using `jwtService.extractUsername()`.`"

9. JWT → Username

Your code:

```
java
String username = jwtService.extractUsername(token);
```

Conceptually:

```
text
JWT
↓
Payload
↓
subject
↓
username
```

Your Day 4 implementation used the JWT `subject` for the username, along with issued-at and expiration claims.

fileciteÖturn0file0ÖL320-L340Ö

For example:

```
json
{
  "sub": "testuser",
  "iat": "...",
  "exp": "..."
```

Then:

```
text
```

```
username = testuser
```

10. Why check `SecurityContext`?

Your code:

```
java
```

```
if (username != null &&  
    SecurityContextHolder.getContext().getAuthentication() == null)
```

There are two checks.

First:

```
java
```

```
username != null
```

The username was successfully extracted.

Second:

```
java
```

```
getAuthentication() == null
```

There isn't already an authentication object in the current `SecurityContext`.

11. 🧠 What is `SecurityContext`?

`SecurityContext` contains authentication information for the current request.

Conceptually:

```
text
```

```

SecurityContext
  ↓
Authentication
  ↓
UserDetails
  ↓
Username + Authorities

```

Your Day 4 notes describe it as the place where Spring Security stores authentication information for the current request.

fileciteÖturn0file0ÖL394-L410Ö

Interview answer

""SecurityContext stores the authentication information for the current request. It allows Spring Security to know which user is authenticated and what authorities that user has.""

12. `SecurityContextHolder`

You used:

```

java

SecurityContextHolder.getContext()

```

Think of it as the mechanism through which your application accesses the current SecurityContext.

Then:

```

java

setAuthentication(authentication)

```

stores the authenticated user's information.

```

java

SecurityContextHolder.getContext()
    .setAuthentication(authentication);

```

Before

```

text

SecurityContext
  ↓

```

```
Authentication = null
```

After

```
text

SecurityContext
  ↓
Authentication
  ↓
testuser
  ↓
ROLE_STAFF
```

Interview answer

""`SecurityContextHolder.getContext().setAuthentication(authentication)` stores the authenticated user's Authentication object in the current SecurityContext so Spring Security can use that authentication and its authorities during authorization.""

13. Loading User Details

Your filter does:

```
java

UserDetails userDetails =
    userDetailsService.loadUserByUsername(username);
```

Flow:

```
text

username
  ↓
CustomUserDetailsService
  ↓
UserRepository
  ↓
MySQL
  ↓
User
  ↓
UserDetails
```

Your Day 4 architecture described this relationship as:

```
text
```

```
Spring Security
↓
UserDetailsService
↓
UserRepository
↓
MySQL
```

fileciteÖturn0file0ÖL212-L227Ö

14. `CustomUserDetailsService`

Your implementation:

```
java

@Override
public UserDetails loadUserByUsername(String username)
    throws UsernameNotFoundException {

    User user = userRepository.findByUsername(username)
        .orElseThrow(() ->
            new UsernameNotFoundException("User not found"));

    return org.springframework.security.core.userdetails.User
        .withUsername(user.getUsername())
        .password(user.getPassword())
        .roles(user.getRole())
        .build();
}
```

It converts your database `User` into Spring Security's `UserDetails`.

15. `.roles(user.getRole())`

This is **very important** for today's authorization topic.

Suppose database:

```
text

username = testuser
role = STAFF
```

Your code:

```
java
.roles("STAFF")
```

results in the Spring Security authority:

```
text
ROLE_STAFF
```

Similarly:

```
text
MANAGER
```

becomes:

```
text
ROLE_MANAGER
```

Remember

```
text

Database
STAFF
  ↓
.roles("STAFF")
  ↓
Spring Security
ROLE_STAFF
```

and:

```
text

Database
MANAGER
  ↓
.roles("MANAGER")
  ↓
Spring Security
ROLE_MANAGER
```

16. JWT Validation

Your filter calls:

```
java

jwtService.validateToken(token, username)
```

Conceptually:

```
text

JWT
↓
validateToken()
↓
Valid?
├─ YES → Create Authentication
└─ NO  → Don't authenticate
```

If validation succeeds, you create the authentication object.

17. Creating the Authentication Object

Your code:

```
java

UsernamePasswordAuthenticationToken authentication =
    new UsernamePasswordAuthenticationToken(
        userDetails,
        null,
        userDetails.getAuthorities());
```

There are three important parts.

First argument

```
java

userDetails
```

Identifies the user.

Second argument

```
java  
null
```

No password is being stored here for this JWT-authenticated request.

Third argument

```
java  
userDetails.getAuthorities()
```

Contains the user's authorities.

For example:

```
text  
ROLE_STAFF
```

or:

```
text  
ROLE_MANAGER
```

18. Why `getAuthorities()` is important

This line:

```
java  
userDetails.getAuthorities()
```

is what allows the Authentication object to carry the user's permissions/roles.

So:

```
text
```

```
User
↓
role = MANAGER
↓
UserDetails
↓
ROLE_MANAGER
↓
Authentication
↓
SecurityContext
↓
Authorization
```

This is the bridge between **authentication** and **role-based authorization**.

19. Setting Authentication Details

Your code:

```
java

authentication.setDetails(
    new WebAuthenticationDetailsSource()
        .buildDetails(request));
```

This attaches details from the current HTTP request to the authentication object.

For today's interview preparation, the more important concept is that the actual authentication object is subsequently stored in the SecurityContext.

20. Storing Authentication

Your most important line:

```
java

SecurityContextHolder.getContext()
    .setAuthentication(authentication);
```

Now Spring Security knows:

```
text
```

```
Current user = testuser
Authority = ROLE_STAFF
```

or:

```
text

Current user = manager
Authority = ROLE_MANAGER
```

21. Continuing the Filter Chain

Finally:

```
java

filterChain.doFilter(request, response);
```

means the current filter has finished its processing and the request should continue through the remaining filter chain.

Conceptually:

```
text

JwtAuthenticationFilter
    ↓
Other Security Filters
    ↓
Authorization
    ↓
Controller
```

22. 🔗 `SecurityFilterChain`

Your `SecurityConfig` contains:

```
java

.addFilterBefore(
    jwtAuthenticationFilter,
    UsernamePasswordAuthenticationFilter.class);
```

This adds your custom JWT filter to Spring Security's filter chain.

Interview answer

"We add the `JwtAuthenticationFilter` to the Spring Security filter chain so that incoming requests containing a Bearer JWT can be processed and authenticated before authorization is performed."

23. What does `addFilterBefore()` mean?

```
java

.addFilterBefore(
    jwtAuthenticationFilter,
    UsernamePasswordAuthenticationFilter.class)
```

Read it as:

"Add `jwtAuthenticationFilter` before `UsernamePasswordAuthenticationFilter` in the security filter chain."

The important concept for your project is:

```
text

Request
↓
JWT Filter
↓
Authentication established
↓
Authorization
```

24. Stateless Authentication

Your `SecurityConfig` has:

```
java

.sessionManagement(session -> session
    .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
```

This means your application uses stateless JWT authentication rather than relying on a traditional server-side authentication session.

Conceptually:

text

```
Request 1 → JWT  
Request 2 → JWT  
Request 3 → JWT
```

Your Day 4 notes define stateless JWT authentication as each request carrying the token needed for authentication rather than relying on a traditional authentication session. `fileciteÖturn0file0ÖL435-L446Ö`

Interview answer

““`SessionCreationPolicy.STATELESS` means the application does not maintain a traditional server-side authentication session. Each request contains the JWT needed to authenticate the user.””

25. Authentication vs Authorization

Authentication

“Who are you?”

Example:

text

```
JWT  
↓  
testuser  
↓  
Valid user  
↓  
Authenticated ✓
```

Authorization

“What are you allowed to do?”

Example:

text

```
testuser  
↓  
ROLE_STAFF  
↓  
POST /api/staff  
↓  
Requires MANAGER
```

↓
Denied ❌

🎤 Interview answer

""Authentication verifies the identity of a user, while authorization determines which resources and operations an authenticated user is allowed to access.""

This distinction was one of the core concepts established in your Day 4 notes. `file:///C:/Users/JohnDoe/Desktop/Notes/Day4/Authentication%20and%20Authorization.md`

26. 🛡️ Role-Based Authorization

Your project now distinguishes:

```
text

STAFF
MANAGER
```

STAFF

Can perform read operations:

```
text

GET /api/staff
GET /api/staff/{id}
GET /api/staff/department
```

MANAGER

Can perform the manager-only write operations:

```
text

POST /api/staff
PUT /api/staff/{id}
DELETE /api/staff/{id}
```

Your actual `StaffController` contains these GET, POST, PUT and DELETE endpoints.

27. `.hasRole()`

Example:

```
java
    .hasRole("MANAGER")
```

This means the authenticated user must have the Manager role.

Because your `UserDetailsService` uses:

```
java
    .roles(user.getRole())
```

Spring Security internally works with:

```
text
ROLE_MANAGER
```

So:

```
java
    .hasRole("MANAGER")
```

matches:

```
text
ROLE_MANAGER
```

Interview answer

""`hasRole()` restricts an endpoint to users having a particular role. In our project, `hasRole("MANAGER")` allows users with the MANAGER role to access the endpoint.""

28. `.hasAnyRole()`

Example:

```
java

.hasAnyRole("STAFF", "MANAGER")
```

means either role is allowed.

```
text

ROLE_STAFF    → ✓
ROLE_MANAGER  → ✓
```

Interview answer

""`hasAnyRole()`` allows access when the authenticated user has at least one of the specified roles.""

29. Your Security Rules

Your authorization configuration is conceptually:

```
text

/auth/register
↓
PUBLIC

/auth/login
↓
PUBLIC

GET /api/staff
↓
STAFF + MANAGER

GET /api/staff/{id}
↓
STAFF + MANAGER

GET /api/staff/department
↓
STAFF + MANAGER

POST /api/staff
↓
MANAGER only

PUT /api/staff/{id}
↓
MANAGER only

DELETE /api/staff/{id}
↓
MANAGER only
```

30. `permitAll()`

Your configuration contains:

```
java

    .requestMatchers(
        "/auth/register",
        "/auth/login")
    .permitAll()
```

These endpoints don't require authentication.

Why?

Users need to be able to:

```
text

Register
  ↓
Login
  ↓
Receive JWT
```

before they can access protected APIs.

31. `.authenticated()`

You also have:

```
java

.anyRequest().authenticated()
```

This means requests not matched by more specific rules must at least have an authenticated user.

Interview answer

""`authenticated()` requires the user to be authenticated before accessing the matched endpoint, but it does not restrict access to a specific role.""

32. Testing You Performed Today

You didn't just configure security—you tested it with Postman.

Test 1 — Valid JWT

```
text

GET /api/staff
+
Bearer valid JWT
```

Result:

```
text

200 OK 
```

Meaning:

```
text

JWT valid
↓
Authentication successful
↓
SecurityContext populated
↓
API accessible
```

33. Test 2 — No JWT

```
text

GET /api/staff
+
No Auth
```

Result:

```
text

403 Forbidden
```

Meaning:

```
text

No authentication
↓
Protected endpoint
↓
Access denied
```

34. Test 3 — Invalid JWT

You modified the JWT and sent it to:

```
text

GET /api/staff
```

Result:

```
text

403 Forbidden
```

Meaning:

```
text

Invalid JWT
↓
Authentication not established
↓
Protected endpoint
↓
Access denied
```

35. Test 4 — STAFF → POST

You created:

```
text

testuser
role = STAFF
```

Then:

```
text

POST /api/staff
```

with the Staff user's JWT.

Result:

```
text

403 Forbidden ❌
```

Why?

```
text

testuser
↓
ROLE_STAFF
↓
POST requires MANAGER
↓
Denied
```

36. Test 5 — MANAGER → POST

You created:

```
text

manager
role = MANAGER
```

Then:

```
text
```

```
POST /api/staff
```

with the Manager's JWT.

Result:

```
text
```

```
201 Created ✓
```

The application successfully created:

```
text
```

```
Manager Test Staff
```

So:

```
text
```

```
manager
↓
ROLE_MANAGER
↓
POST /api/staff
↓
hasRole("MANAGER")
↓
Allowed
↓
StaffController
↓
StaffService
↓
Database
↓
201 Created
```

🔥 This is the strongest practical demonstration of today's topic.

37. Test 6 — STAFF → DELETE

You also tested the Staff user against:

```
text
```

```
DELETE /api/staff/12
```

and it was rejected.

Therefore:

text

STAFF → DELETE ❌
MANAGER → DELETE ✅

This demonstrates role-based authorization beyond just the POST operation.

38. ★ Most Important Flow to Memorize

For a protected request:

text

```
Client
↓
Authorization: Bearer JWT
↓
JwtAuthenticationFilter
↓
Extract JWT
↓
Extract username
↓
CustomUserDetailsService
↓
UserRepository
↓
Database
↓
UserDetails + Authorities
↓
Validate JWT
↓
Create Authentication
↓
SecurityContext
↓
Spring Security Authorization
↓
Check Role
↓
Controller
```

Short version:

"JWT → Filter → Authentication → SecurityContext → Authorization → Controller"

Your Day 4 notes explicitly highlighted this security progression as the target milestone. NfileciteÖturn0file0ÖL614-L632Ô

39. 🎤 Top Spring Security Interview Questions & Answers

Q1. What is Spring Security?

"Spring Security is a framework used to handle authentication and authorization in Spring applications."

Q2. What is authentication?

"Authentication verifies the identity of a user."

Example:

```
text

Username + Password
    ↓
Verify credentials
    ↓
User authenticated
```

Q3. What is authorization?

"Authorization determines which resources and operations an authenticated user is allowed to access."

Q4. Authentication vs Authorization?

"Authentication verifies who the user is, while authorization determines what the authenticated user is allowed to do."

Q5. What is JWT?

"JWT stands for JSON Web Token. It is used to represent authentication information and allows the client to authenticate subsequent requests to protected APIs."

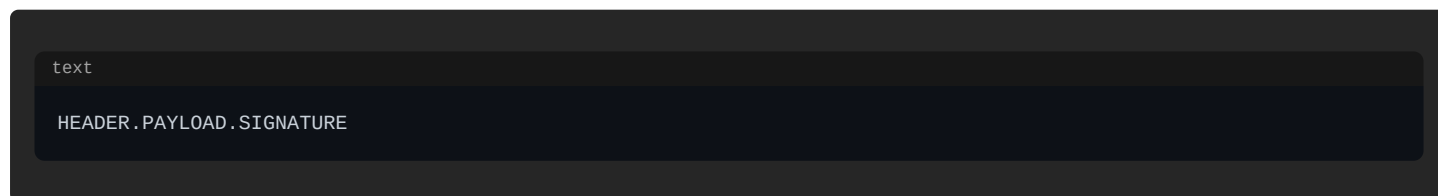
Your Day 4 notes describe JWT as representing authentication information and being sent with subsequent protected requests.

Q6. Why do we use JWT?

"JWT allows the client to authenticate subsequent requests without sending the username and password with every request."

Q7. What are the three parts of JWT?

"Header, Payload, and Signature."



Q8. What is the JWT signature?

"The JWT signature helps verify the authenticity and integrity of the token and helps ensure that it has not been modified."

This was one of your Day 4 interview answers. [NfileciteÖturn0file0ÖL341-L346Ö](#)

Q9. Why does JWT have an expiration time?

"Expiration limits how long a token can be used. This reduces the risk if a token is stolen or compromised."

Q10. What is a Bearer token?

"Bearer is the authentication scheme used in the Authorization header to indicate that the following value is the authentication token."

Q11. What is `JwtAuthenticationFilter``?

"It is a custom security filter that extracts the JWT from the Authorization header, validates it, loads user details, creates Authentication, and stores it in the SecurityContext."

Q12. Why extend `OncePerRequestFilter``?

"We extend `OncePerRequestFilter`` to implement JWT processing logic that runs once for the request."

Q13. Why use `substring(7)`?

"It removes the `Bearer` prefix from the Authorization header and gives us only the JWT token."

Q14. Does `substring(7)` extract the username?

"No. It extracts the JWT. The username is extracted separately using `jwtService.extractUsername(token)`."

🔥 This was one of the points you corrected during today's quiz.

Q15. What is `UserDetailsService`?

"`UserDetailsService` loads a user's authentication information, usually by username, for Spring Security."

Q16. What is `CustomUserDetailsService`?

"It is our custom implementation of `UserDetailsService` that retrieves the user from the database through `UserRepository` and converts it into Spring Security's `UserDetails`."

Q17. What is `AuthenticationManager`?

"`AuthenticationManager` coordinates the authentication process and verifies the user's credentials."

Your Day 4 notes describe this as the component responsible for performing authentication and returning an authenticated result when credentials are valid. `fileciteÖturn0file0ÖL246-L264Ö`

Q18. What is `SecurityContext`?

"`SecurityContext` stores authentication information for the current request, allowing Spring Security to know the authenticated user and their authorities."

Q19. What does `SecurityContextHolder` do?

"`SecurityContextHolder` provides access to the `SecurityContext` associated with the current security context, allowing us to store the `Authentication` object."

Q20. Why do we call `setAuthentication()`?

"We call `setAuthentication()` to store the authenticated user's Authentication object in the SecurityContext so Spring Security can use it for authorization."

Q21. Why use `addFilterBefore()`?

"It adds our JWT authentication filter to the Spring Security filter chain before the specified filter, allowing JWT authentication processing to happen as part of security processing."

Q22. What is `SecurityFilterChain`?

"`SecurityFilterChain` defines how Spring Security processes HTTP requests, including authentication, authorization and security filters."

Q23. Why use `SessionCreationPolicy.STATELESS`?

"It configures the application to use stateless authentication, where the server doesn't rely on a traditional authentication session and each request carries its JWT."

Q24. What is an authority?

"An authority represents a permission or role associated with an authenticated user that Spring Security can use when making authorization decisions."

Q25. What is a role?

"A role is a type of authority used to represent a user's level of access, such as STAFF or MANAGER."

Q26. What does `.roles(user.getRole())` do?

"It assigns the user's database role to the Spring Security UserDetails. Spring Security represents it with the `ROLE_` prefix internally."

Example:

```
text
```

```
STAFF → ROLE_STAFF  
MANAGER → ROLE_MANAGER
```

Q27. What does `` .hasRole("MANAGER")`` mean?

"It allows access only to authenticated users who have the MANAGER role."

Q28. What does `` .hasAnyRole("STAFF", "MANAGER")`` mean?

"It allows access to users who have either the STAFF or MANAGER role."

Q29. What happens if a STAFF user accesses a MANAGER-only endpoint?

"The user is authenticated if the JWT is valid, but authorization fails because the user does not have the required MANAGER role. Spring Security denies access."

Q30. Is the STAFF user authenticated in that situation?

"Yes. Authentication can succeed while authorization fails. The user is known and authenticated, but they don't have permission for that operation."



Very important interview concept.

Q31. Does Spring Security wait for a Manager to approve the STAFF request?

"No. In our role-based authorization setup, there is no approval workflow. Spring Security checks the user's role immediately and denies the request if the required role is missing."

Q32. Why is ``/auth/login`` permitted without authentication?

"Because the user needs to log in before they have a JWT. The login endpoint authenticates the credentials and generates the JWT."

Q33. Why is `/api/staff` protected?

"Because it contains application data and operations that should only be available to authenticated users with the appropriate permissions."

Q34. What happens when a valid JWT is sent?

"The JWT filter extracts and validates the token, loads the user details, creates Authentication, stores it in the SecurityContext, and Spring Security can then authorize the request."

Q35. What happens when no JWT is sent?

"The JWT filter does not establish authentication. Since the protected endpoint requires authentication, Spring Security denies the request."

Q36. What happens when an invalid JWT is sent?

"The JWT fails validation, so the filter does not establish valid authentication. The protected request is then denied by Spring Security."

40. ★ Most Important Interview Question

"Explain how you implemented JWT authentication and role-based authorization in your Spring Boot project."

Interview-ready answer

"In my Spring Boot project, I implemented JWT-based authentication using Spring Security. During login, the user provides their username and password. The AuthenticationManager uses UserDetailsService to load the user from the database, and PasswordEncoder verifies the password against the BCrypt hash. If authentication succeeds, JwtService generates a JWT and returns it to the client."

For subsequent protected requests, the client sends the JWT in the Authorization header using the Bearer scheme. I created a custom JwtAuthenticationFilter by extending OncePerRequestFilter. The filter extracts the token, extracts the username, loads UserDetails, validates the JWT, creates a UsernamePasswordAuthenticationToken with the user's authorities, and stores it in the SecurityContext."

I then configured this filter in the SecurityFilterChain using addFilterBefore. After authentication is established, Spring Security performs authorization based on the user's role. In my project I have STAFF and MANAGER roles. STAFF can perform read operations, while manager-only operations such as adding, updating and deleting staff require the MANAGER role. I

tested the implementation using Postman, where a STAFF user was denied access to manager operations while a MANAGER user was successfully allowed."

🔥 This is the answer you should practice repeatedly.

41. ★ Complete Project Flow — Interview Diagram

If the interviewer asks you to explain your implementation on a whiteboard:



42. 🧠 10 Lines to Memorize Before an Interview

If you have very little time before an interview, remember these:

text

1. Authentication = Who are you?
2. Authorization = What can you do?
3. JWT = authentication token for subsequent requests.
4. Bearer JWT is sent in Authorization header.
5. JwtAuthenticationFilter extracts and validates JWT.
6. UserDetailsService loads user information.
7. Authentication contains user + authorities.
8. SecurityContext stores Authentication for the current request.
9. hasRole() checks a specific role.
10. Valid authentication does NOT automatically mean authorization.

43. 🏆 Day 5 Achievement

Today you moved from **understanding JWT conceptually** to actually proving the complete security mechanism in your application.

Day 4 ended here:

text

JWT Filter 🟡
Protected Staff APIs ⌚
Role Authorization ⌚

Day 5 ends here:

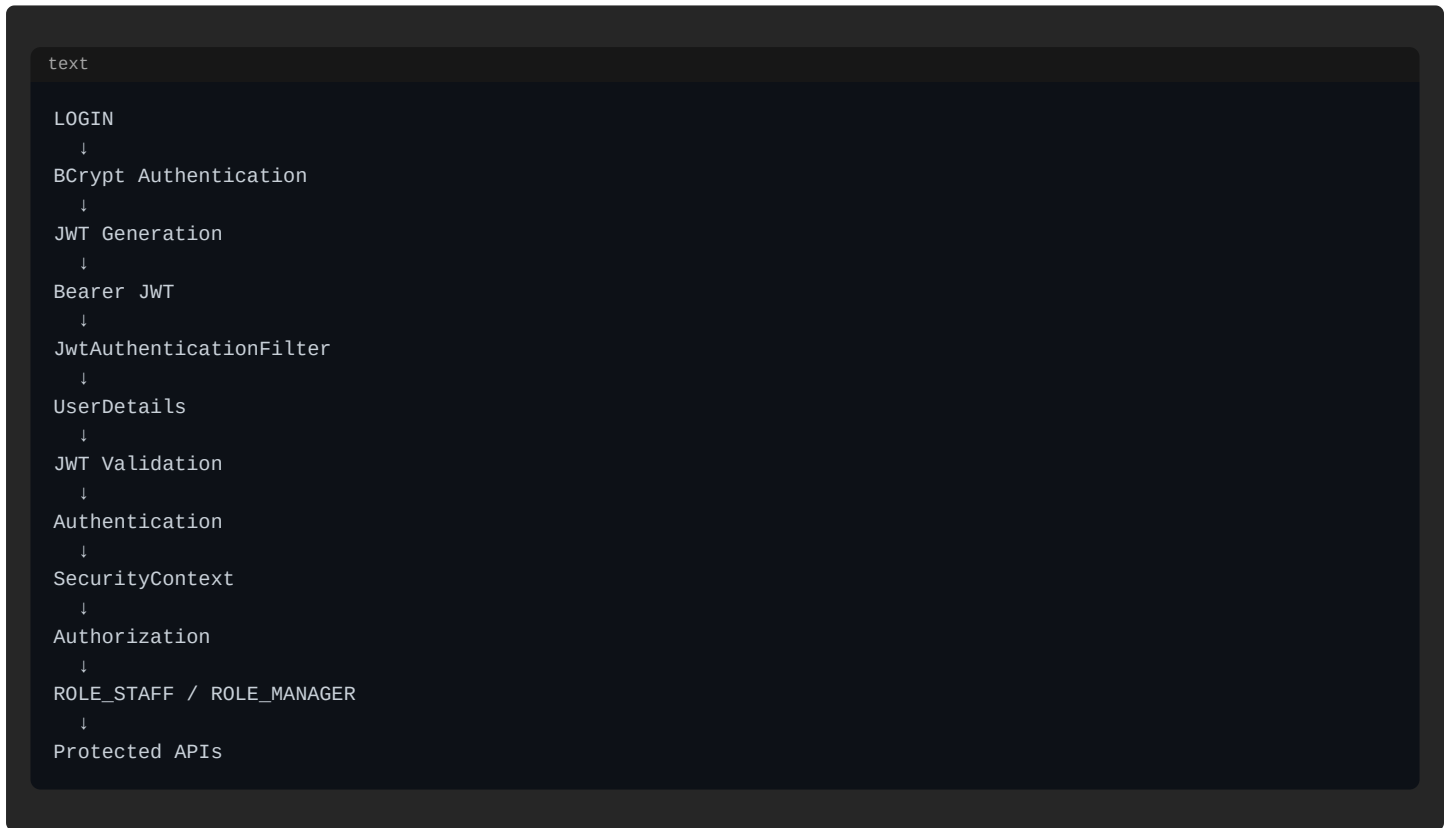
text

JWT Filter	✓
SecurityFilterChain	✓
Protected API	✓
SecurityContext	✓
JWT validation	✓
STAFF role	✓
MANAGER role	✓
Role authorization	✓
STAFF denied	✓
MANAGER allowed	✓
Postman testing	✓

Your original Day 4 notes identified JWT Filter, Protected Staff APIs and Role-Based Authorization as the remaining security milestone. NfileciteÖturn0file0ÖL561-L613Ö

Final Day 5 milestone

You can now explain and demonstrate:



Day 5 Spring Security + JWT + Role-Based Authorization:  COMPLETED. 

Tomorrow, you should **not restart Spring Security**. Your next study session should move forward from this completed security milestone according to your one-month preparation plan.